Notes on the numerical solution of convection-diffusion equations

April 30, 2026

Content

1 Problem formulation **1**
1.1 Galerkin finite elements 2

2 The FEM solver **3**
2.1 Solution of the linear system 3
2.2 Dirichlet boundary conditions 4
2.3 Implementation of the streamline upwing stabilization term 4

3 Sample solutions **4**

1 Problem formulation

The general convection-diffusion-reaction (or advection-diffusion-reaction) equation governs, for example, the fate of a contaminant dissolved in a fluid moving with velocity $q(x,t)$ in a domain $\Omega \subset \mathbb{R}^d$, $d = 1, 2, 3$ and undergoing a linear kinetic transformation with constant $c(x)$. It is given by:

$$\frac{\partial u}{\partial t} = \operatorname{div}(D\,\nabla u) - \operatorname{div}(\beta u) + cu + f \qquad \in \Omega \times (0,T) \qquad (1.1)$$

$$u(x,0) = u_0(x) \qquad x \in \Omega, t = 0 \qquad (1.2)$$

$$u(x,t) = u_D(x,t) \qquad x \in \Gamma_D, t > 0 \qquad (1.3)$$

$$D\,\nabla u \cdot \vec{n} = q_N(x,t) \qquad x \in \Gamma_N, t > 0 \qquad (1.4)$$

$$(\beta u + D\,\nabla c) \cdot \vec{n} = q_C(x,t) \qquad x \in \Gamma_C, t > 0 \qquad (1.5)$$

where:

- u: [-] solute concentration (generally normalized to 1);
- t [T]: time;
- x: [L] spatial coordinate with respect to a Eulerian reference system (for $d = 2$:: $x = (x_1, x_2)$);
- K [L^2/T]: diffusion coefficient (assumed a scalar function of (x,t));
- β [L/T]: velocity field;
- f [1/T]: forcing function;
- $\Omega \subset \mathbb{R}^d$ domain ($d = 1, 2,$ or 3);
- $u_0(x)$: initial concentration;
- $\Gamma = \Gamma_D \cup \Gamma_N \cup \Gamma_C$ boundary of Ω;
- Γ_D: Dirichlet boundary;
- $u_D(x,t)$: value of the Dirichlet boundary condition;
- Γ_N: Neuman boundary;
- $q_N(x,t)$: value of the diffusive flux at Neuman boundary;
- Γ_C: Cauchy boundary;
- $q_C(x,t)$: value of total flux at Cauchy boundary.

In this project, we want to solve the convection-diffusion equation (Eq. (1.1)) in the stationary case ($\partial u/\partial t = 0$) and in absence of reaction ($c = 0$). The equation is defined in the domain Ω given by:

$$\Omega = \{(x, y) : 0 \leq x \leq 1, 0 \leq y \leq 1\}$$

with constant diffusion and velocity, with zero forcing and no reaction. The data and boundary conditions are appropriately chosen so that a boundary layer occurs close to the boundary at $y = 1$. Thus we want to solve:

$$-D\Delta u + \beta \cdot \nabla u = 0 \qquad \text{in } \Omega \tag{1.6}$$

$$u(x, t) = 1 \qquad x \in \Gamma_1, t > 0 \tag{1.7}$$

$$u(x, t) = 0 \qquad x \in \Gamma_2, t > 0 \tag{1.8}$$

where the boundaries Γ_1 and Γ_2 are given by:

$$\Gamma_1 = \{(x, y) : [x = 0, 0 \leq y \leq 1] \cup [0 \leq x \leq 0.3, y = 0]\}$$

$$\Gamma_2 = \{(x, y) : [x = 1, 0 \leq y \leq 1] \cup [0 \leq x \leq 1, y = 1] \cup [0.3 \leq x \leq 1, y = 0]\}$$

Note that there is no closed-formed solution for this problem. Thus we want to find the $\mathcal{P}_1$-FEM numerical solution on a given triangular mesh.

1.1 Galerkin finite elements

Let u_h be the approximate FEM solution expanded on the basis functions of the search space $\mathcal{V}_h$:

$$u_h(x, y) = \sum_{i=1}^{n} u_i \phi_i (x, y) \tag{1.9}$$

where $\phi_i (x, y)$, $i = 1, \ldots, n$ are piecewise linear Lagrangian basis functions and u_i are the values of u_h at the n nodal points of the triangulation. Subsituting u_h into Eq. (1.6) we have, after application of Green's Lemma:

$$\int_\Omega \left(D\frac{\partial u_h}{\partial x}\frac{\partial \phi_i}{\partial x} + D\frac{\partial u_h}{\partial y}\frac{\partial \phi_i}{\partial y} \right) \mathrm{d}x - \int_{\Gamma_N} \left(D\frac{\partial u_h}{\partial x}\nu_x + D\frac{\partial u_h}{\partial y}\nu_y \right) \phi_i \, \mathrm{d}s + \int_\Omega \left(\beta_x \frac{\partial u_h}{\partial x} + \beta_y \frac{\partial u_h}{\partial y} \right) \phi_i \, \mathrm{d}x = 0 \qquad i = 1, \ldots, n\,. \tag{1.10}$$

Using Eq. (1.9) and the boundary conditions we obtain the following linear system:

$$(A + B)\mathbf{u} = \mathbf{q} \tag{1.11}$$

where:

$$A = \{a_{ij}\} \qquad a_{ij} = \int_\Omega \left[\left(D \frac{\partial \phi_j}{\partial x} \frac{\partial \phi_i}{\partial x} + D \frac{\partial \phi_j}{\partial y} \frac{\partial \phi_i}{\partial y} \right) \right] \, \mathrm{d}x$$

$$B = \{b_{ij}\} \qquad b_{ij} = \int_\Omega \left(\beta_x \frac{\partial \phi_j}{\partial x} + \beta_y \frac{\partial \phi_j}{\partial y} \right) \phi_i \, \mathrm{d}x$$

are stiffness and transport matrices, respectively. Note that now $B \neq B^T$.
We use the standard Lagrangian basis functions, and build these matrices using the standard element-by-element assembly procedure.

2 The FEM solver

The FEM solver shares the same structure of the code developed for the stationary diffusion equations in the first project. The most important difference is that the transport matrix is nonsymmetric. This fact affects possibly the computational cost of the assembly process and certainly the of the linear system solver. Methods taylored to sparse non symmetric systems automatically by the "backslash" of MATLAB or by appropriate native MATLAB routines that implement the preconditioned GMRES or BiCGSTAB methods.
In addition, some form of stabilization is needed to avoid oscillations in the numerical solution when convection domninates. We elect to use here the stabilizing Streamline Upwind diffusion term.

2.1 Solution of the linear system

Algorithm 1 Prototype call to the GMRES Matlab function with the ILU Preconditioner.

```
1: restart=10; tol=1e-9; maxit=20;
2: setup.type=nofill';
3: setup.milu='off';
4: [L,U]=ilu(SYSMAT,setup);
5: x=gmres(SYSMAT,RHS,restart,tol,maxit,L,U);
```

The system matrix $H + B$ is sparse and has the same sparsity structure of the stiffness matrix H. As said above, the "backslash" MATLAB method can be used or, alternatively, one could use the GMRES algorithm, as implemented in MATLAB, preconditioned by the incomplete LU factorization. The call to the MATLAB GMRES function is provided in algorithm **??**. Note that, for GMRES to work in Matlab, SYSMAT must be of type "sparse".

2.2 Dirichlet boundary conditions

Note that the use of the penalty method to impose the boundary conditions will in this case cause convergence problems to the GMRES iteration. To see this, we try with a large value of the penalty parameter ($\epsilon = 10^{-40}$) and verify that the boundary condition is correctly imposed, but the real residual ($\|b - Ax_k\|$) becomes very large, indicating that the solution is not correct. This is due to cancellations that occur when calculating the exit norm $\|r_k\| / \|b\|$. Actually, with such a value for λ no iterations are performed, since the relative residual is smaller than the requested tolerance for any initial condition (λ cancels in the evaluation of r_k but $\|b\| \approx \lambda$. Hence the boundary lifting operator needs to be employed, or, alternatively but less accurately, a relatively small value of ϵ (e.g. $\approx 10^{-10}$) needs to be employed.
If you use GMRES instead of MATLAB "backslash" you may want to experiment with different values of the penalty parameter.

2.3 Implementation of the streamline upwing stabilization term

The streamline upwind diffusion elemental matrix is given by:

$$a_h(u_h, v_h) = a(u_h, v_h) + \delta(\beta \cdot \nabla u_h, \beta \cdot \nabla v_h) = a(u_h, v_h) + a_s(u_h, v_h),$$

with the stabilization matrix S given on element T by:

$$s_{ij}^T = \delta \int_T \left(\beta_x^{(T)} \frac{\partial \phi_i}{\partial x} + \beta_y^{(T)} \frac{\partial \phi_i}{\partial y}\right)\left(\beta_x^{(T)} \frac{\partial \phi_j}{\partial x} + \beta_y^{(T)} \frac{\partial \phi_j}{\partial y}\right) \mathrm{d}x$$

norm of the diffusion tensor assumed constant in each element, h_T is the length of the largest edge of the triangle T, where the term δ is an empirical coefficient used to calibrate the desired amount of numerical diffusion to be added.

3 Sample solutions

The student is asked to implement the solver as described before and to apply it to the solution of the described problem using the provided mesh. It is easier to start from a copy of a Poisson linear FEM solver. To verify the implementation, first try the code with $v_x = v_y = 0$ and compare its solution with the solution obtained with the available Poisson solver and then explore what is happening for different values of β. The student is also asked to explore the unstabilized and the stabilized numerical solution for values of the diffusion coefficient D variying in the interval $[10^{-3}, 1]$ with a constant velocity field $\vec{v} = (1, 3)^T$. Find the minimum value below which oscillations occur, and determine a "pseudo" Peclet number for the considered mesh. Then, it is asked to apply the code with the SUD stabilization and explore its behavior varying D and δ.
In Figs. 3.1 and 3.2 sample solutions are shown as examples.

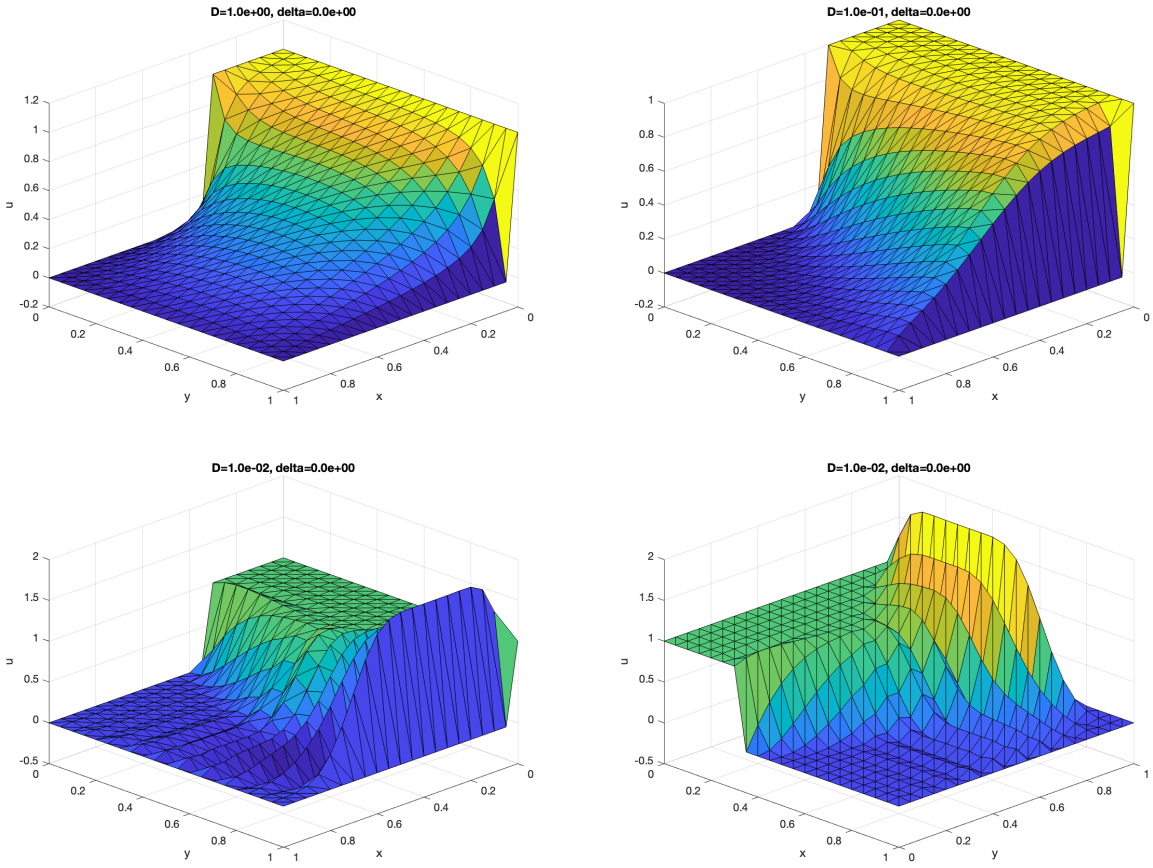


FIGURE 3.1: *Convection-diffusion solution with linear Galerkin without stabilization. The top panel shows the cases with* $D = 1$ *and* $D = 10^{-1}$*. The bottom panel show the ossillations appearing when* $D = 10^{-2}$*. Two different viewports are used to better appreciate the behavior of the numerical solution.*

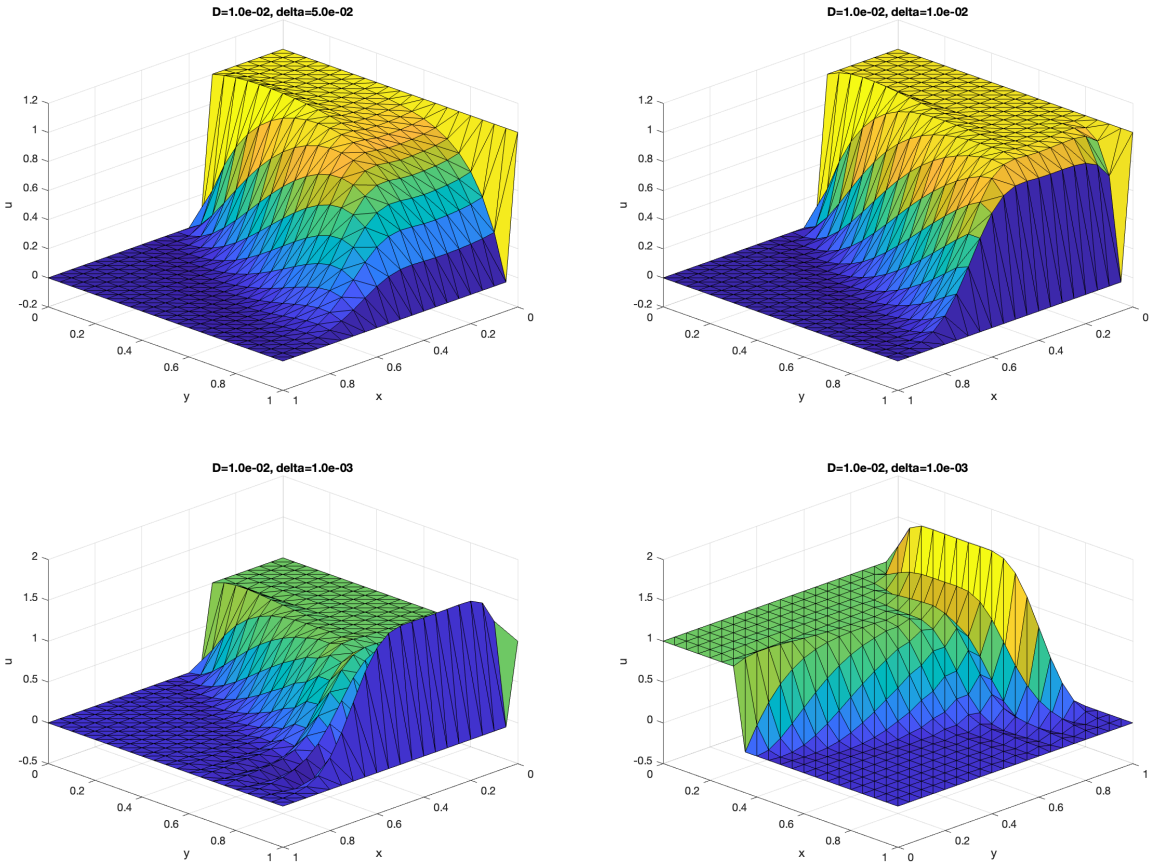


FIGURE 3.2: *Convection-diffusion solution with linear Galerkin with Streamline-Diffusion stabilization for* $D = 10^{-2}$. *The upper panel shows the solution with* $\delta = 5 \times 10^{-2}$ *and* $\delta = 10^{-2}$, *while the lower panel shows the solution with* $\delta = 10^{-3}$ *using two different viewports. Note that with larger* δ *the numerical oscillations decrease but numerical diffusion increases.*